wanted the version that *runs on one board, every day, with a real agent on top of it.* Assumes you can install Linux, read YAML, and follow a shell script. That is the whole barrier, and it is a fair one.

---

1. The shape of a 24/7 agent

Most people try to run an agent the way a chatbot runs: one long-lived session that gets bigger and slower until it dies. The shape that survives is the opposite.

The loop. A small supervisor script starts a fresh agent process on a fixed cadence — here, every cycle gets a clean context window, a briefing (mission, deadlines, yesterday's log), a wall-clock budget, and one job: *check the inbox, check the channel, take the top backlog item, finish it, record it, stop.* The loop waits a few minutes and starts the next one. The agent's entire life is a few dozen file edits and tool calls inside a 90-minute budget; then it is gone and a fresh one starts from whatever the disk says.

Why a fresh context every cycle is cheaper than one giant session. This is measured, not philosophy. A compacted session forces the model to re-read its whole prompt from zero — on this box that costs roughly **5.5 ms per re-stated token**, so a session that has accumulated ~1 million tokens of restatement pays ~90 minutes of wall in pure re-reading before doing any work. A fresh cycle starts with a ~170-450k token briefing instead. The per-day numbers from the box's own cost ledger (last two days: 1.40 M vs 171 k prefill tokens, -87.7%) show the difference is not marginal — it is the difference between "works" and "crawls."

The five-part cycle, in order:

1. **Orient** — read the inbox, the channel to the owner, the backlog, the deadlines. 2. **Size** — estimate the unit in minutes and check the estimate against the wall budget *before starting*. A 40-minute unit needs ~66 minutes of headroom at the 70% line; if it doesn't have that, it does not start. This one rejection rule (estimated > ~60% of remaining budget → park, don't push) is what separates a 24/7 box that finishes work from one that dies mid-unit with nothing on disk. 3. **Work** — one unit. Log as you go, not at the end: each meaningful step lands in the activity log immediately so a crash mid-way still leaves a trail. 4. **Verify** — a fix is "verified" only when the *exact command that reported the defect* runs the good result after the change. A sub-command, a `--verify` flag, or "the related thing looks faster" is evidence about a component, not about the defect. 5. **Close, then stop.** The close is one atomic call that writes: the journal line, the owner's notebook line, the plain-language status line (for the human watching a dashboard), and the done flag — in *one* write, because a crash between writes is what strands a cycle (work done, records missing, next cycle re-derives). Then the agent ends its turn. Filling the full budget is not a goal; stopping at "done" is.